This project focused on building a Boggle solver and then improving it through code review and linting instead of just submitting the first working version. The solver takes in a grid and a dictionary and returns the words that can actually be formed on the board. It uses depth-first search (DFS) to move through the grid in all directions while making sure the same cell is not reused in a single word. It also handles cases like “Qu” tiles, where one position represents more than one character, which required adjusting how the index moves through each word.

The initial version of the solver worked correctly, but the feedback I received pointed out several areas where the code could be improved. The overall structure made sense, especially with the use of helper functions like exists and dfs, but there were a few things that could be cleaned up. One issue was an unused import that didn’t need to be there. Some conditions, like the boundary check in the DFS function, could be written in a more compact and readable way. There were also places where the logic worked but wasn’t clearly explained, such as converting words to uppercase before comparing them to the grid. Adding comments in those spots made the code easier to follow.

When reviewing my partner’s code, I focused on both what was working and what could be improved. I pointed out areas where the structure or logic was clear, and also mentioned places where naming or comments could be improved. Instead of directly saying something needed to be changed, I framed things more as suggestions or questions. That made the feedback feel less harsh and more like a normal collaboration.

After getting feedback, I updated my solver by removing anything unnecessary, adding a few clarifying comments, and fixing formatting issues. A lot of the work was making sure the code followed PEP 8, especially indentation, spacing, and line length. Some lines were too long and had to be split up, and there were small spacing issues throughout the

file. These changes didn't affect how the solver worked, but they made it easier to read and maintain.

Running pycodestyle in Codio showed a number of style errors in both the solver and the test file. Most of them were related to inconsistent indentation, extra whitespace, and formatting problems. I fixed these by standardizing everything to four spaces, cleaning up comments, and removing trailing spaces. The test file had a lot of similar issues, so I fixed those as well, but only in terms of formatting. The actual test logic and expected outputs were not changed.

After making these updates, I ran the provided test suite again. All of the tests passed, including the ones that check edge cases and larger boards, so the functionality stayed the same. This confirmed that the changes were only improving readability and not breaking anything in the solver.

This project showed that even when code is already working, there is still a lot that can be improved. Small things like formatting and comments make a big difference when someone else has to read the code. The review process helped catch things that I overlooked, and the linter pointed out issues that are easy to miss. It also gave a better idea of how code is actually reviewed and improved in a real development setting, where readability and consistency matter just as much as correctness.